

○ Start & Select Buttons

Hardware Overview

○ What are they?

The **START (S2_B)** and **SELECT (S3_B)** buttons are simple tactile switches. They are Digital Inputs.
*(Note: These two buttons work exactly the same way. In this documentation, we focus on the **Start Button**, but everything applies to the **Select Button** too - just replace the word '**start**' with '**select**' in your code!)*

⚡ Electrical Explanation

Buttons seem simple, but electrically they need help to be reliable. On the K-Comp, these buttons are wired as **Active HIGH**.

1. **Idle (Not Pressed):** The button is open. The microcontroller reads **0V (LOW)**.
2. **Pressed:** The switch closes. The microcontroller reads **5V (HIGH)**.

Key Hardware Facts:

- **Start Connection:** Wired to **Digital Pin D3**.
- **Signal Flow:** 5V → Switch → Resistor 220Ω → Pin D3

Software & Programming

 `kcomp_button_start.h` (for the **Select** button, use `kcomp_button_select.h`)

Method A: "Polling" (The Simple Check)

This is the easiest way. You simply ask the K-Comp: *'Is the button pressed right now?'*

- **Best for:** checking if a button is held down (like an accelerator pedal).
- **Downside:** It might miss fast clicks or count one click as many (bouncing).

Method B: "Event-Driven" (The Advanced Manager)

Instead of checking constantly, you say: *'Here is a function. Run it **ONLY** when you detect a click.'*

- **Best for:** Menu navigation, shooting, or toggles.

Function Reference

Function Name	Description & Parameters
<code>void initButtonStart()</code>	Setup: Configures Pin D3. Called once in setup() .
<code>bool isButtonStartPressed()</code>	Polling Method: Returns <code>true</code> if the button is currently held down, <code>false</code> otherwise.
<code>void pollStartButton()</code>	The Manager: Checks hardware, filters noise, and triggers events. Must be called continuously in <code>loop()</code> for Method B to work.
<code>void attachButtonStartClick(func)</code>	Event Setup: Registers <code>func</code> to run on a Single Click .
<code>void attachButtonStartDoubleClick(func)</code>	Event Setup: Registers <code>func</code> to run on a Double Click .
<code>void attachButtonStartLongPressed(func)</code>	Event Setup: Registers <code>func</code> to run when the button is pressed .

⚠ Pitfall: "Bouncing"

When pressing a button, metal contacts vibrate. A single press might look like 10 fast the processor.

- **Method A (Polling)** sees all the noise.
- **Method B (Events)** filters ("Debouncing").

🚀 Implementation Guide: Using Events

Method B (Events), requires 3 steps. Then your code can react to inputs without freezing.

- Step 1 (Define the Action): Create a `void` function that contains the code you want to run.

```
void myJumpAction() { Serial.println("JUMP!"); }
```

- Step 2 (Attach the Action) In `setup()`, tell the button manager to use your function.

```
void setup() {  
    initButtonStart();  
    // Connect the "myJumpAction" function to the Click event  
    attachButtonStartClick(myJumpAction);  
}
```

- Step 3 (Run the Manager) In `loop()`, keep the manager running so it can watch the hardware. Make sure that `pollStartButton()` is called at least like every 200ms.

```
void loop() { pollStartButton(); }
```

📄 Sample Program

This program demonstrates the Event-Driven approach. It handles **Single Clicks**, **Double Clicks**, and **Long Presses** simultaneously.

```
#include <kcomp_button_start.h>  
  
// --- Step 1: Define Your Action Functions ---  
void onSingleClick() { Serial.println("Action: Single Click!"); }  
void onDoubleClick() { Serial.println("Action: Double Click!"); }  
void onLongPress() { Serial.println("Action: Button Held Down..."); }  
  
void setup() {  
    Serial.begin(115200);  
    initButtonStart(); // Initialize Hardware  
  
    // --- Step 2: Attach Functions to Events ---  
    // Note: Write the function name WITHOUT "()" brackets here!  
    attachButtonStartClick(onSingleClick);  
    attachButtonStartDoubleClick(onDoubleClick);  
    attachButtonStartLongPressed(onLongPress);  
  
    Serial.println("System Ready. Click the button!");  
}  
  
void loop() {  
    // --- Step 3: Run the Manager ---  
    // You must call this frequently!  
    pollStartButton();  
  
    // You can do other work here, and the button will still respond.  
    delay(10);  
}
```

💡 **Try it out:** You can find the full sample program **SimpleSelectButton** and **SelectButtonCallbackFunctions** in the K-Comp examples folder!